Transform Your Future with
"HELLO WORLD!"
INSTITUTE

Why learn with us ?

Training From Experts

One - on – One Guidance

100% Practical

Real-Time Projects & Case Studies

Placement Preparation Support

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Contact :- 8085896740 @ hello_world_institute Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Python List

A list in Python is used to store the sequence of various types of data. Python lists are mutable type its mean we can modify its element after it created. However, Python consists of six data-types that are capable to store the sequences, but the most common and reliable type is the list.

A list can be defined as a collection of values or items of different types. The items in the list are separated with the comma (,) and enclosed with the square brackets [].

Ex:

```
list = ["apple", "banana", "cherry"]
print(list)
```

List Items

- List items are ordered, changeable, and allow duplicate values.
- List items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

- When we say that lists are ordered, it means that the items have a defined order, and that order will not change.
- If you add new items to a list, the new items will be placed at the end of the list.

Note: There are some list methods that will change the order, but in general: the order of the items will not change.

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates

Since lists are indexed, lists can have items with the same value:

Ex : Lists allow duplicate values:

```
list1 = ["apple", "banana", "cherry", "apple", "cherry"]

print(list1)
```

List Length : To determine how many items a list has, use the len() function:

Ex :

```
list = ["apple", "banana", "cherry"]
print(len(list))
```

List Items - Data Types

Ex : String, int and Boolean data types:

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
 list3 = [True, False, False]
```

A list can contain different data types:

Ex: A list with strings, integers and Boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

type()

From Python's perspective, lists are defined as objects with the data type 'list':
<class 'list'>

The list() Constructor

It is also possible to use the list() constructor when creating a new list.

OFFLINE CLASSES
ONLINE LECTURE
RECORDED ACCESS

Ex : Using the list() constructor to make a List:

```
list1 = list(("apple", "banana", "cherry")) # note the double round-brackets

 print(list1)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

Note:

*Set *items* are unchangeable, but you can remove and/or add items whenever you like.

**As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

Access Items

List items are indexed and you can access them by referring to the index number:
Ex : Print the second item of the list:

```
list = ["apple", "banana", "cherry"]
print(list[1])
```

Negative Indexing

Negative indexing means start from the end -1 refers to the last item, -2 refers to the second last item etc.

Ex : Print the last item of the list:

```
list = ["apple", "banana", "cherry"]
print(list[-1])
```

Range of Indexes

- You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new list with the specified items.

Ex:

```
list = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]

print(list[2:5])
```

Ex:

```
list = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]

print(list[:4])
```

Ex:

```
list = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]

print(list[2:])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list:
Ex:

```
list = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]

print(list[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a list use the in keyword:

Ex: Check if "apple" is present in the list:

```
list1 = ["apple", "banana", "cherry"]

 if "apple" in list1:

print("Yes, 'apple' is in the fruitslist")
```

Change Item Value

To change the value of a specific item, refer to the index number:

Ex: Change the second item:

```
list1 = ["apple", "banana", "cherry"]

list1[1] = "blackcurrant"

print(list1)
```

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

Ex:

```
list1 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
list1[1:3] = ["blackcurrant", "watermelon"]

 print(list1)
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Ex: Change the second value by replacing it with *two* new values:

```
list1 = ["apple", "banana", "cherry"]

 list1[1:2] = ["blackcurrant", "watermelon"]

 print(list1)
```

Note: The length of the list will change when the number of items inserted does not match the number of items replaced.

If you insert *less* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Ex: Change the second and third value by replacing it with *one* value:

```
list1 = ["apple","banana","cherry"]

list1[1:3] = ["watermelon"]
print(list1)
```

Insert Items

- To insert a new list item, without replacing any of the existing values, we can use the insert() method.
- The insert() method inserts an item at the specified index:

Ex: Insert "watermelon" as the third item:

```
list1 = ["apple","banana","cherry"]
list1.insert(2, "watermelon")
print(list1)
```

Add List Items

1. **Append Items** : To add an item to the end of the list, use the append() method

   Ex: Using the append() method to append an item:

```
list1=["apple","banana","cherry"]
list1.append("orange")
print(list1)
```

2. **Insert Items** :

   - To insert a list item at a specified index, use the insert() method.
   - The insert() method inserts an item at the specified index:

Ex: Insert an item as the second position:

```
list1 = ["apple","banana","cherry"]
list1.insert(1, "orange")
print(list1)
```

3. **Extend List** : To append elements from *another list* to the current list, use the extend() method.

Ex: Add the elements of tropical to thislist:

```
list1 = ["apple", "banana", "cherry"]
tropical=["mango","pineapple""pap]
list1.extend(tropical)

 print(list1)
```

The elements will be added to the *end* of the list.

Add Any Iterable

The extend() method does not have to append *lists*, you can add any iterable object (tuples, sets, dictionaries etc.).

Ex: Add elements of a tuple to a list:

```
list1 = ["apple","banana","cherry"]
tuple = ("kiwi", "orange")
list1.extend(tuple)

print(list1)
```

Remove List Items

1. **Remove Specified Item** : The remove() method removes the specified item.

Eg:

```
Remove "banana":

list1= ["apple", "banana","cherry"]
list1.remove("banana")
print(list1)
```

2. <u>Remove Specified Index</u> : The pop() method removes the specified index.

Ex: Remove the second item:

```
list1 = ["apple","banana","cherry"]
list1.pop(1)

print(list1)
```

If you do not specify the index, the pop() method removes the last item.

Ex: Remove the last item:

```
list =
["apple","banana","cher]
list.pop()

print(list)
```

The del keyword also removes the specified index:

Ex: Remove the first item:

```
list1 =
["apple","banana""cherry]
del list1[0]

print(list1)
```

The del keyword can also delete the list completely.

Ex: Delete the entire list:

```
list1 = ["apple","banana","cherry"]
del list1
```

3. Clear the List

- The clear() method empties the list.
- The list still remains, but it has no content.

Ex: Clear the list content:

```
list1 = ["apple", "banana", "cherry"]
list1.clear()
 print(list1)
```

Loop Lists : You can loop through the list items by using a for loop:

Eg:

```
list1 = ["apple", "banana",
"cherry"] for x in list1:   print(x)
```

1. Loop Through the Index Numbers

- You can also loop through the list items by referring to their index number.
- Use the range() and len() functions to create a suitable iterable.

Eg :

```
list1 = ["apple", "banana", "cherry"]
for i in range(len(list1)):
  print(list1[i])
```

The iterable created in the example above is [0, 1, 2].

2. Using a While Loop

- You can loop through the list items by using a while loop.
- Use the len() function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.
- Remember to increase the index by 1 after each iteration

Eg:

```
list1 = ["apple", "banana", "cherry"]
i = 0
while i < len(list1):
 print(list1[i])
 i = i + 1
```

3. Looping Using List Comprehension

List Comprehension offers the shortest syntax for looping through lists:

Ex : A short hand for loop that will print all items in a list:

```
list1 = ["apple","banana","cherry"]
[print(x) for x in list1]
```

List Comprehension

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Syntax :

```
newlist = [expression for item in iterable if condition == True]
```

The return value is a new list, leaving the old list unchanged.

Eg: Based on a list of fruits, you want a new list, containing only the fruits with the letter "a" in the name.
Without list comprehension you will have to write a for statement with a conditional test inside:

Eg:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []
for x in fruits:
  if "a" in x:
    newlist.append(x)
  print(newlist)
```

With list comprehension you can do all that with only one line of code:

Eg:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if "a" in x]


 print(newlist)
```

Condition

The *condition* is like a filter that only accepts the items that valuate to True.

Example

Only accept items that are not "apple":

```
newlist = [x for x in fruits if x != "apple"]
```

The condition if x != "apple" will return True for all elements other than "apple", making the new list contain all fruits except "apple".

Eg.With no if statement:

```
newlist = [x for x in fruits]
```

Iterable : The *iterable* can be any iterable object, like a list, tuple, set etc.

Eg.You can use the range() function to create an iterable:

```
newlist = [x for x in range(10)]
```

Same example, but with a condition:

Eg:

```
Accept only numbers lower than 5:

newlist = [x for x in range(10) if x < 5]
```

Sort Lists

1. Sort List Alphanumerically

List objects have a sort() method that will sort the list alphanumerically, ascending, by default: **Eg.Sort the list alphabetically:**

```
list1 = ["orange", "mango", "kiwi", "pineapple", "banana"]
list1.sort()

 print(list1)
```

Eg.Sort the list numerically:

```
list1 = [100, 50, 65, 82, 23]
list1.sort()
print(list1)
```

2. **Sort Descending :** To sort descending, use the keyword argument reverse = True:

Eg. Sort the list descending:

```
list1 = ["orange", "mango", "kiwi", "pineapple", "banana"]
list1.sort(reverse = True)

 print(list1)
```

Eg. Sort the list descending:

```
list1 = [100, 50, 65, 82, 23]
list1.sort(reverse = True)
print(list1)
```

Reverse Order : The reverse() method reverses the current sorting order of the elements.

Eg. Reverse the order of the list items:

```
list1 = ["banana", "Orange", "Kiwi", "cherry"]
list1.reverse()

print(list1)
```

Copy a List

- You cannot copy a list simply by typing list2 = list1, because: list2 will only be a *reference* to list1, and changes made in list1 will automatically also be made in list2.
- There are ways to make a copy, one way is to use the built-in List method copy().

Eg. Make a copy of a list with the copy() method:

```
list1 = ["apple", "banana", "cherry"]
mylist = list1.copy()
print(mylist)
```

Eg. Make a copy of a list with the list() method:

```
list1 = ["apple", "banana", "cherry"]
mylist = list(list1)
print(mylist)
```

Join Two Lists

- There are several ways to join, or concatenate, two or more lists in Python.
- One of the easiest ways are by using the + operator.

Eg.Join two list:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)
```

Another way to join two lists is by appending all the items from list2 into list1, one by one:

Eg. Append list2 into list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]
for x in list2:
  list1.append(x)
print(list1)
```

Or you can use the extend() method, which purpose is to add elements from one list to another list:

Eg. Use the extend () method to add list2 at the end of list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
 print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list

Contact :- 8085896740 @ hello_world_institute Hello World Institute
Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

OFFLINE CLASSES
ONLINE LECTURE
RECORDED ACCESS